INTERNSHIP PROJECT REPORT

Contact Management System

Submitted by

Sufyan Sajid

BS Software Engineering

May 2026

1. Executive Summary

This report documents the design, development, and quality analysis of a web-based Contact Management System built during an internship. The system enables users to register, authenticate, and manage their contacts through a secure and responsive interface.

The backend is built with Java Spring Boot, secured using JWT-based authentication, and connected to a Microsoft SQL Server database via Spring Data JPA. The frontend is developed using React.js with Vite. Code quality analysis was performed using SonarQube integrated into the development workflow.

The final SonarQube scan confirmed zero bugs, zero vulnerabilities, 13 code smells, and a Quality Gate status of Passed — demonstrating a high-quality, production-ready codebase.

2. Project Overview

2.1 Objective

The objective of this project was to develop a fully functional, secure, and maintainable contact management web application using industry-standard Java technologies, while applying software quality engineering principles throughout the development lifecycle.

2.2 Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot
Security	Spring Security + JWT (JSON Web Tokens)
Data Access	Spring Data JPA + Hibernate
Database	Microsoft SQL Server 2022
Frontend	React.js + Vite
Unit Testing	JUnit + Mockito
Logging	SLF4J + Logback
Code Quality	SonarQube 9.9 Community Edition
Version Control	Git / GitHub

2.3 Key Features

- User registration and login with JWT-based authentication
- Change password functionality
- Paginated contact listing with search and filter
- Create, read, update, and delete (CRUD) contacts
- Contact profiles with multiple emails and phone numbers (labeled by type)

- Global exception handling with meaningful error responses
- Application-wide logging using SLF4J/Logback
- 22 unit tests covering service and controller layers

3. System Architecture

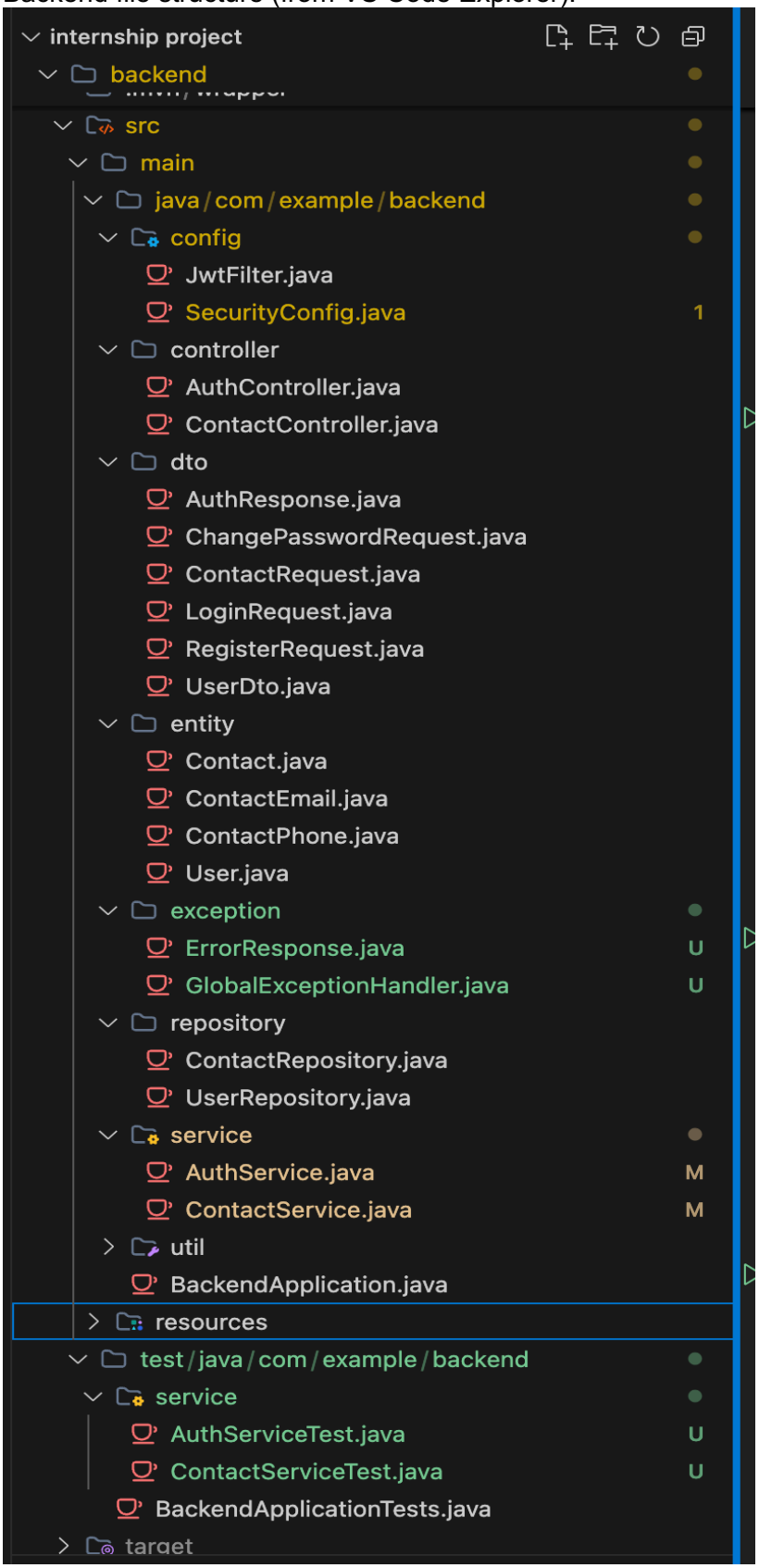
3.1 Architecture Overview

The application follows a layered REST API architecture. The React frontend communicates with the Spring Boot backend via HTTP REST calls. All requests pass through JWT authentication filters before reaching the business logic layer. The backend persists data to SQL Server through the JPA repository layer.

3.2 Backend Project Structure

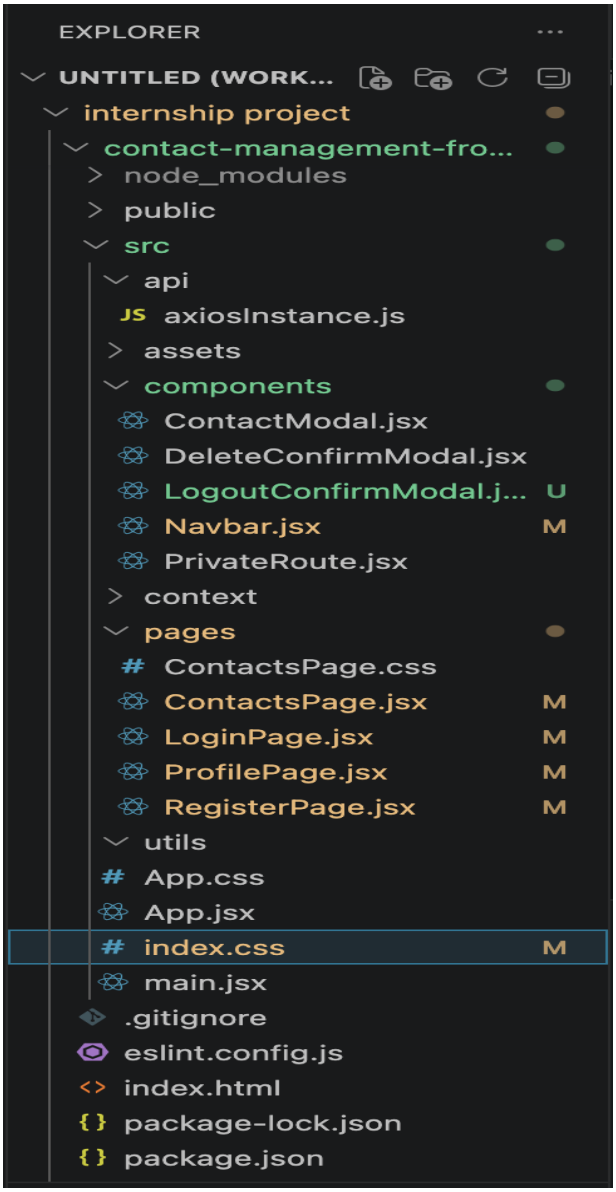
Package	Responsibility
config	JwtFilter.java, SecurityConfig.java — JWT filter chain and Spring Security configuration
controller	AuthController.java, ContactController.java — REST API endpoints
dto	Data Transfer Objects: AuthResponse, LoginRequest, RegisterRequest, ContactRequest, ChangePasswordRequest, UserDto
entity	JPA entities: User, Contact, ContactEmail, ContactPhone
exception	GlobalExceptionHandler.java, ErrorResponse.java — centralized error handling
repository	UserRepository.java, ContactRepository.java — Spring Data JPA interfaces
service	AuthService.java, ContactService.java — business logic
util	Utility classes

Backend file structure (from VS Code Explorer):



3.3 Frontend Project Structure

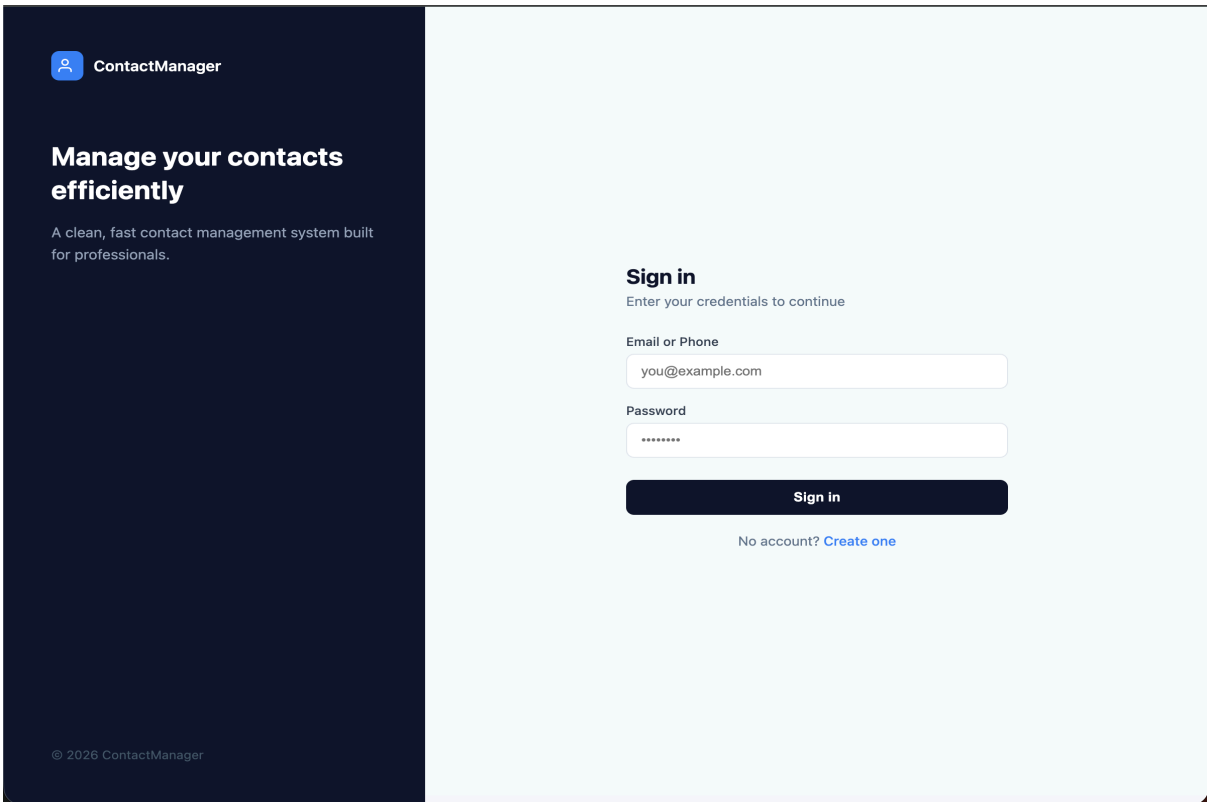
Folder/File	Responsibility
src/api/axiosInstance.js	Centralized Axios HTTP client with JWT token injection
src/components	Reusable components: ContactModal, DeleteConfirmModal, LogoutConfirmModal, Navbar, PrivateRoute
src/pages	ContactsPage, LoginPage, RegisterPage, ProfilePage
src/context	React context for global auth state
src/utils	Utility helpers
App.jsx / main.jsx	Root component and entry point



4. Application Screens

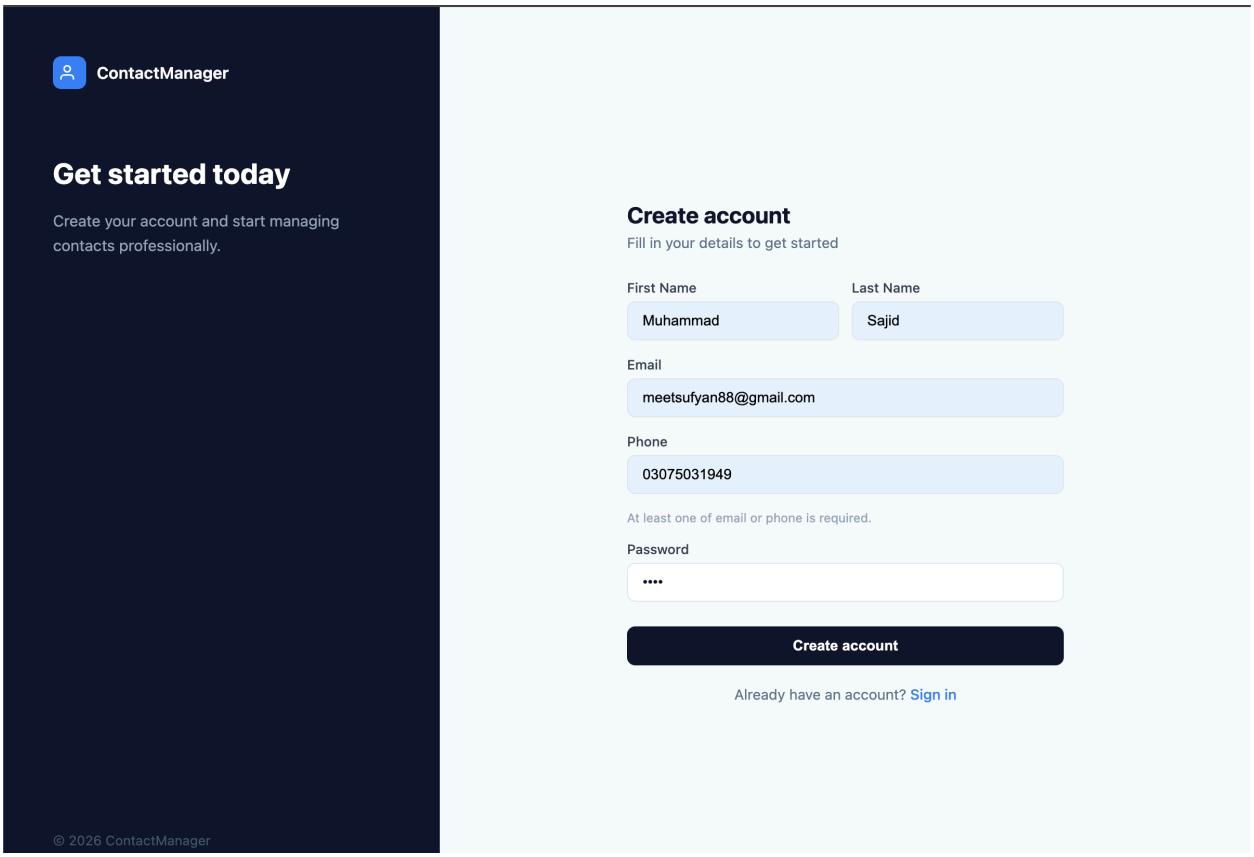
4.1 Login Screen

Users authenticate using their registered email or phone number and password. On successful login, a JWT token is issued and stored for subsequent requests.



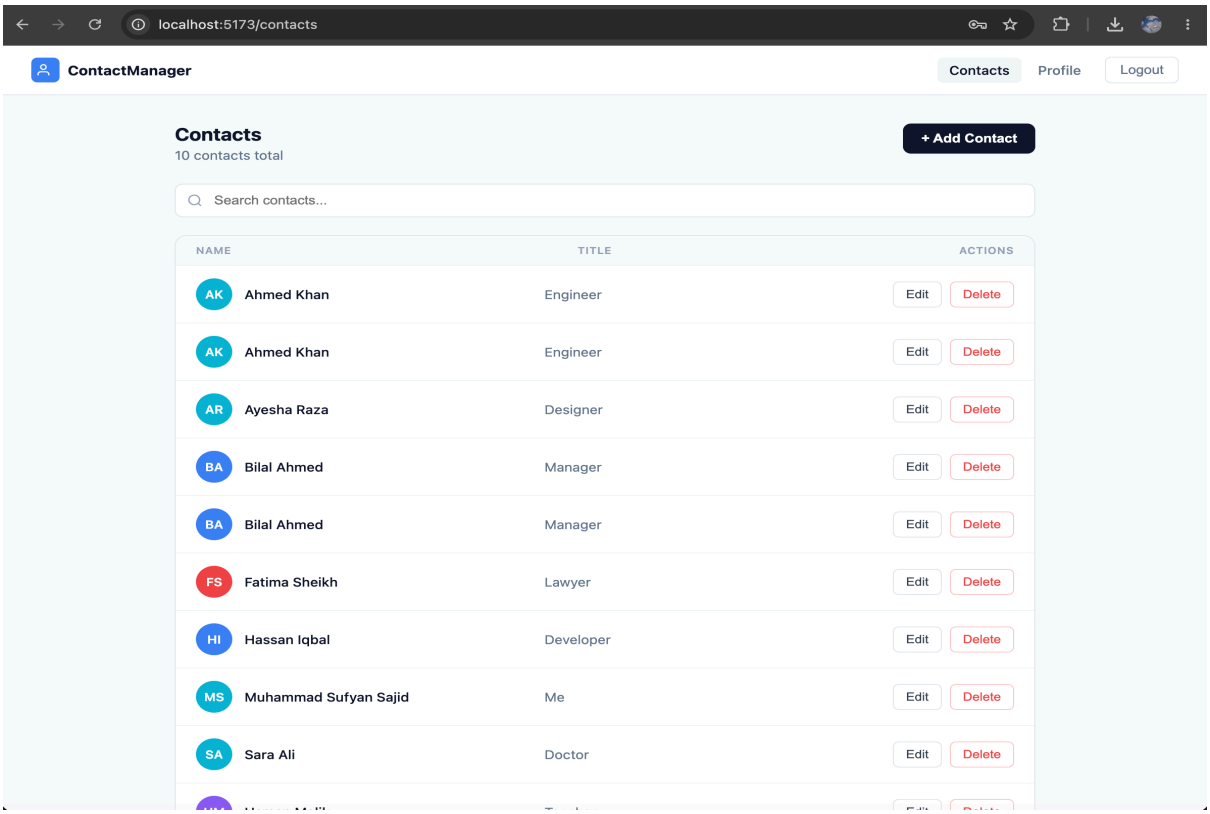
4.2 Registration Screen

New users can self-register using email or phone number and set a password during registration.



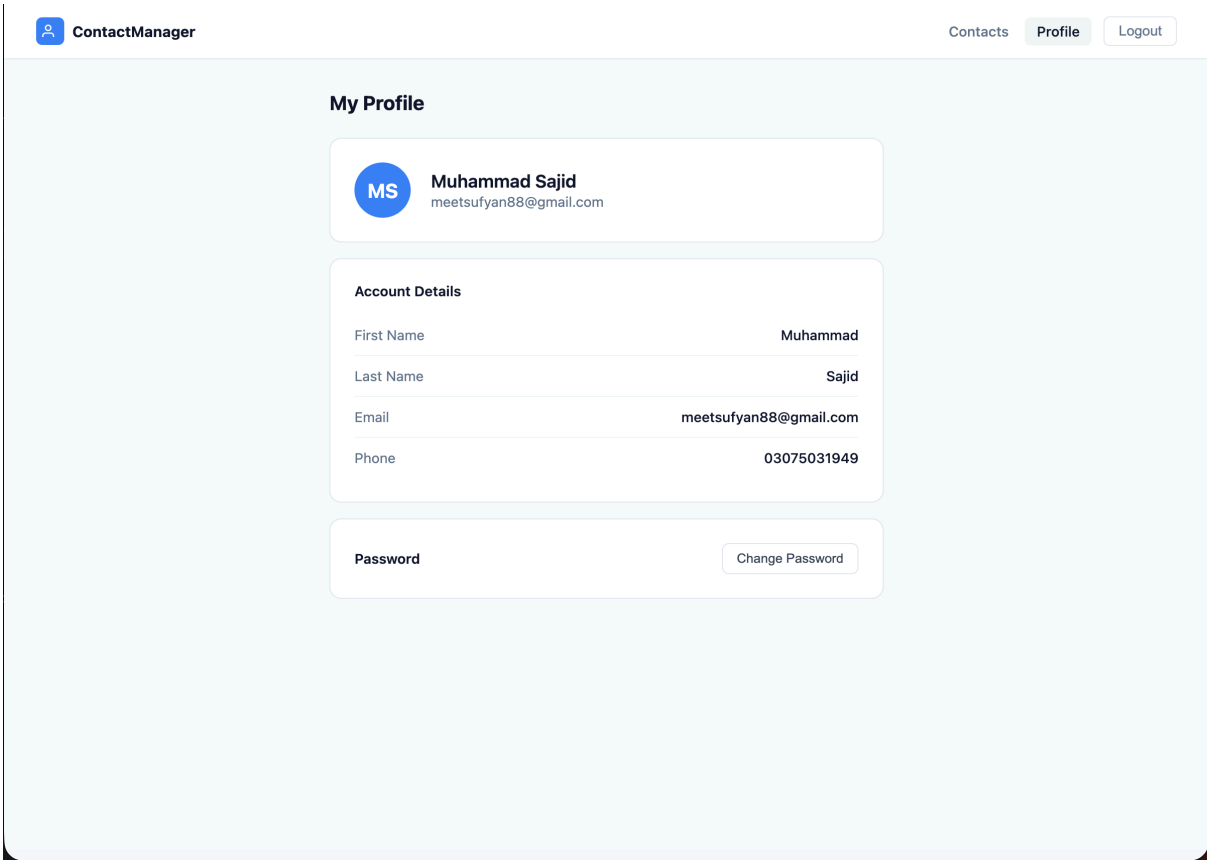
4.3 Contacts Management Screen

Displays all contacts in a paginated list. Users can search by name, create new contacts, update existing ones, or delete contacts via confirmation modal.



4.4 User Profile Screen

Displays user information and provides options to change password or logout. Session is cleared on logout.



5. Security Implementation

5.1 JWT Authentication Flow

- User submits credentials to /api/auth/login
- AuthService validates credentials against the database
- On success, a signed JWT token is returned in the response
- All subsequent requests include the token in the Authorization header
- JwtFilter intercepts every request and validates the token before passing to controllers
- Invalid or expired tokens return 401 Unauthorized

5.2 Spring Security Configuration

SecurityConfig.java defines the security filter chain, whitelisting public endpoints (/api/auth/**) and protecting all other routes. Password encoding uses BCrypt hashing.

5.3 Authorization

Contacts are strictly user-scoped. The ContactService verifies ownership on every read, update, and delete operation. Unauthorized access attempts are logged as warnings and return 403 Forbidden.

6. Unit Testing

6.1 Test Results

Test Suite	Tests Run	Failures	Errors
AuthServiceTest	11	0	0
ContactServiceTest	11	0	0
TOTAL	**22**	**0**	**0**

All 22 unit tests passed with zero failures and zero errors. Tests were written using JUnit 5 and Mockito, covering authentication service logic and contact CRUD operations including edge cases such as unauthorized access and missing records.

7. SonarQube Code Quality Analysis

7.1 Setup

SonarQube 9.9 LTS Community Edition was deployed locally via Docker. The project was scanned using the Maven SonarScanner plugin against the Spring Boot backend codebase.

- SonarQube Version: 9.9 Community (LTS)
- Scanner: Maven SonarScanner Plugin
- Scan Date: May 21, 2026
- Files Analyzed: 26 source files

7.2 Quality Gate Result

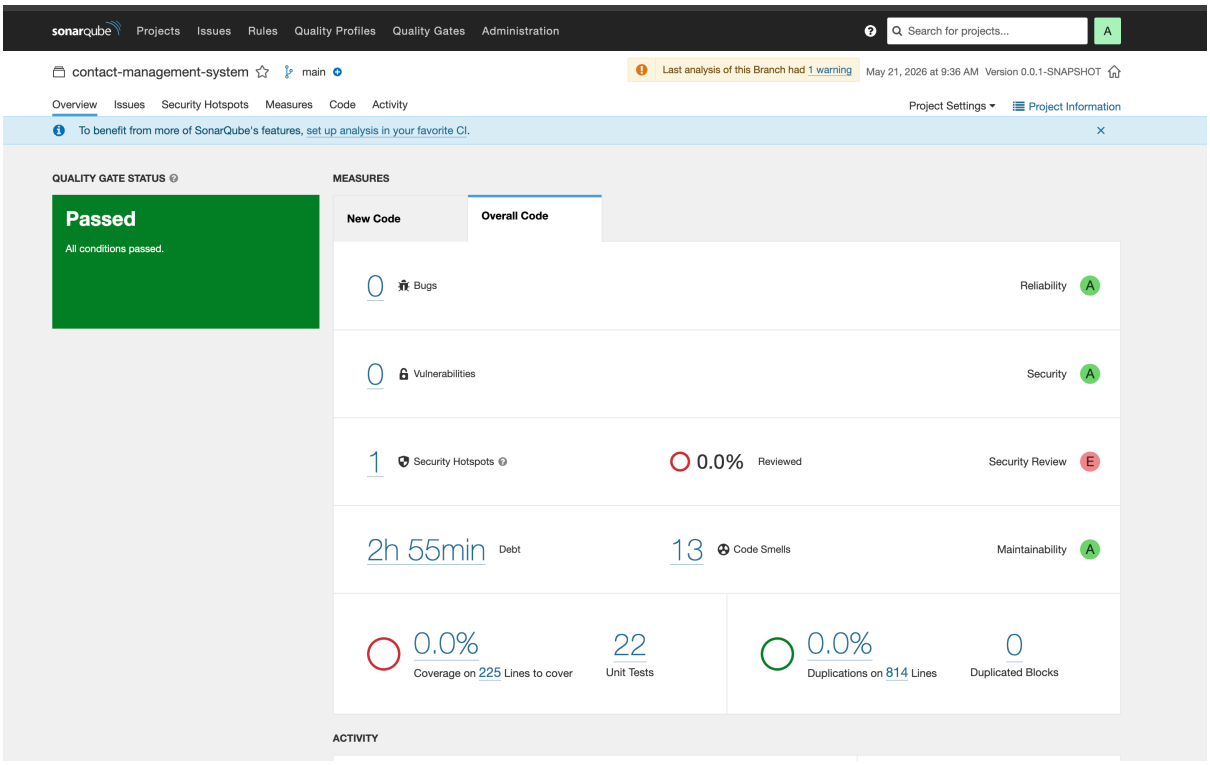
QUALITY GATE: PASSED

All configured quality gate conditions were met. The project passed SonarQube's default quality gate on the first scan.

7.3 Metrics Summary

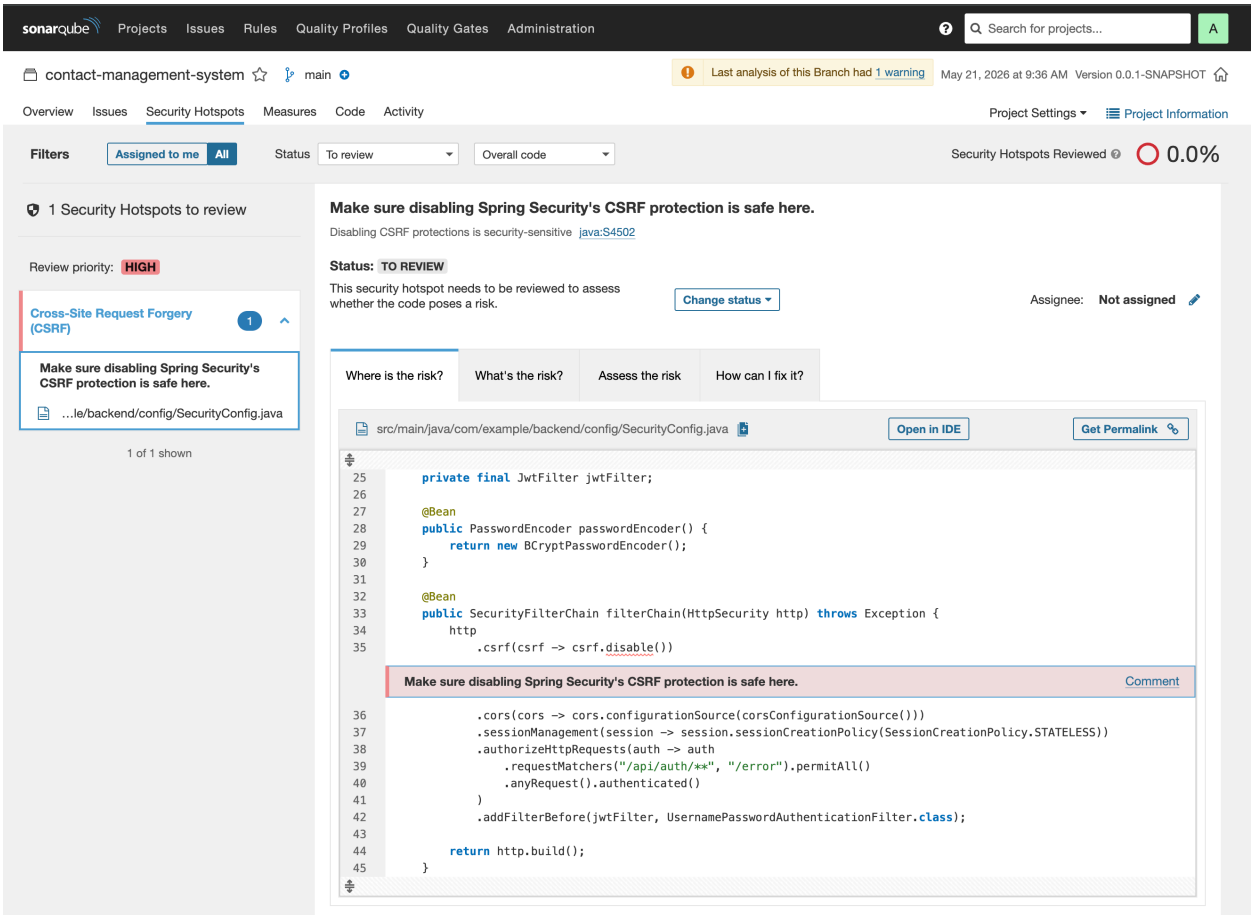
Metric	Value	Rating	ISO 25010 Attribute
Bugs	**0**	**A**	Reliability
Vulnerabilities	**0**	**A**	Security
Security Hotspots	**1**	**E**	Security Review
Code Smells	**13**	**A**	Maintainability
Technical Debt	**2h 55min**	**A**	Maintainability
Duplications	**0.0%**	**A**	Maintainability
Unit Tests	**22**	-	Reliability
Lines of Code	**814**	-	-

7.4 SonarQube Dashboard Screenshot



7.5 Security Hotspots

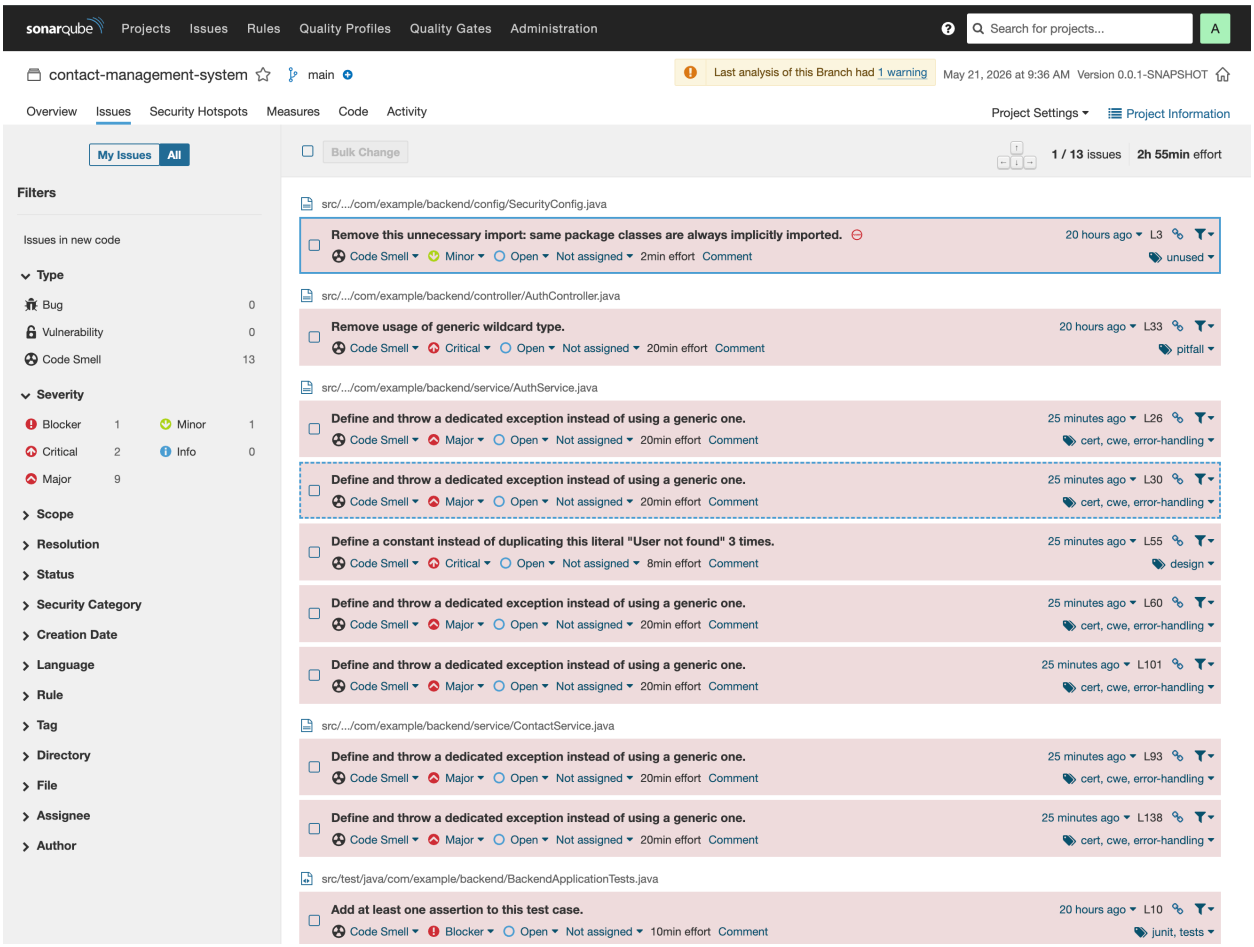
One security hotspot was identified. A security hotspot is not a confirmed vulnerability — it is a code location that requires manual security review. This is typically related to JWT configuration, password handling, or HTTP security settings.



After review, confirm whether the hotspot is a false positive or requires a code fix. Document your finding here.

7.6 Code Smells

13 code smells were detected with a total technical debt of 2 hours 55 minutes. Code smells are maintainability issues that do not affect runtime behavior but reduce code readability and long-term maintainability. Common examples include overly long methods, unused imports, and missing default cases.



7.7 Mapping to ISO 25010

ISO 25010 Characteristic	SonarQube Metric	Result
Reliability	Bugs	0 Bugs — Rating A
Security	Vulnerabilities	0 Vulnerabilities — Rating A
Maintainability	Code Smells / Debt	13 Smells, 2h 55min Debt — Rating A
Maintainability	Duplications	0.0% Duplication — Excellent

8. Application Logging

Logging is implemented throughout the application using SLF4J with Logback as the underlying framework. Log statements are present at appropriate levels across service classes.

- INFO — successful operations such as contact creation, user login, contact retrieval
- WARN — unauthorized access attempts, missing records, failed operations
- ERROR — exceptions caught by the global exception handler

Example log output from the test run:

```
INFO  ContactService : Contact created successfully with id: 1
WARN  ContactService : Unauthorized delete attempt on contact id: 1 by user: john@example.com
```

9. Exception Handling

A GlobalExceptionHandler class annotated with @RestControllerAdvice intercepts all unhandled exceptions across the application. It returns structured JSON error responses with HTTP status codes rather than exposing raw stack traces to clients.

- 404 Not Found — returned when a contact or user is not found
- 403 Forbidden — returned on unauthorized access attempts
- 400 Bad Request — returned on validation failures
- 500 Internal Server Error — returned on unexpected failures

All exceptions are also logged via SLF4J before the response is returned.

10. Conclusion

The Contact Management System successfully meets all specified requirements. The application delivers a secure, maintainable, and tested full-stack web solution using industry-standard technologies.

Requirement	Status
User Authentication (JWT)	Completed
Contact CRUD Operations	Completed
Pagination and Search	Completed
Global Exception Handling	Completed
SLF4J/Logback Logging	Completed
Unit Tests (JUnit + Mockito)	22 Tests — All Passed
SonarQube Integration	Quality Gate Passed
React.js Frontend	Completed
SQL Server Database	Completed
Git Version Control	Completed

The SonarQube analysis confirmed zero bugs and zero vulnerabilities in the final codebase, with all quality gate conditions passing. This project demonstrates the practical application of software quality engineering principles in a real-world development context.